

Reinforcement Learning I

Giulio Turrisi, Postdoctoral Researcher
Italian Institute of Technology

- Motivation
- Problem Definition
- Tabular Methods
- Deep Learning

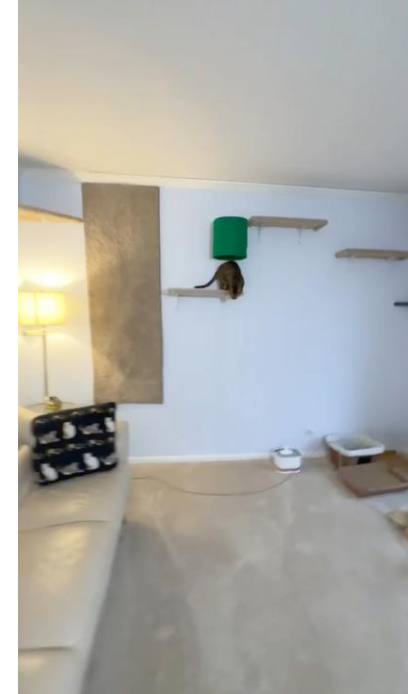
This lecture is inspired by Pieter Abbeel's Reinforcement Learning series:

<https://www.youtube.com/watch?v=2GwBez0D20A&list=PLwRJQ4m4UJjNymuBM9RdmB3Z9N5-0IIY0>

- **Optimal control** often relies on a predefined contact/gait sequence to keep the optimization tractable
- Solving an optimal control problem online can become expensive for high-dimensional, contact-rich systems.
- Accurate models of the robot and its interaction with the environment **are difficult to obtain and scale**.



Grandia et al, 2023



Instead of modeling every situation explicitly, **can we learn from interaction and experience?**

A Few RL Highlights

2013 - DQN (Deepmind)

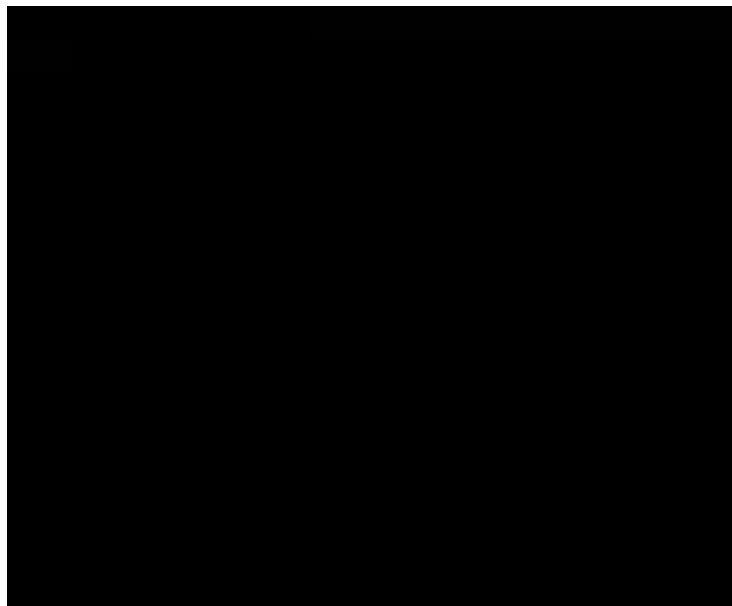


“Playing Atari with Deep Reinforcement Learning”, Mnih et al, 2013

A Few RL Highlights

2013 - DQN (Deepmind)

2015 - DDPG (Deepmind)



“Continuous control with deep reinforcement learning”, Lillicrap et al, 2015

A Few RL Highlights

2013 - DQN (Deepmind)

2015 - DDPG (Deepmind)

2017 - PPO (Berkley)



“Proximal Policy Optimization Algorithms”, Schulman et al, 2017

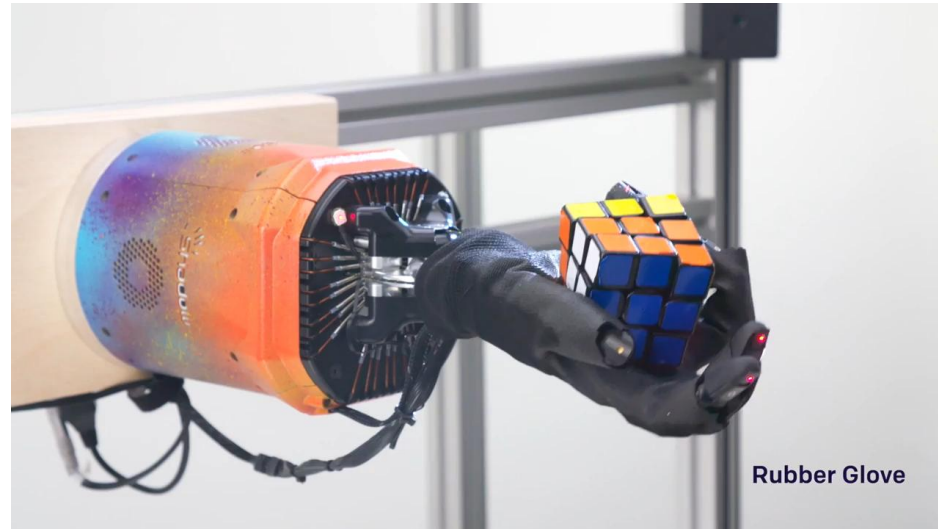
A Few RL Highlights

2013 - DQN (Deepmind)

2015 - DDPG (Deepmind)

2017 - PPO (Berkley)

≥ 2017 – Real Robots



Rubber Glove

“Solving Rubik's Cube with a Robot Hand”, OpenAI, 2019

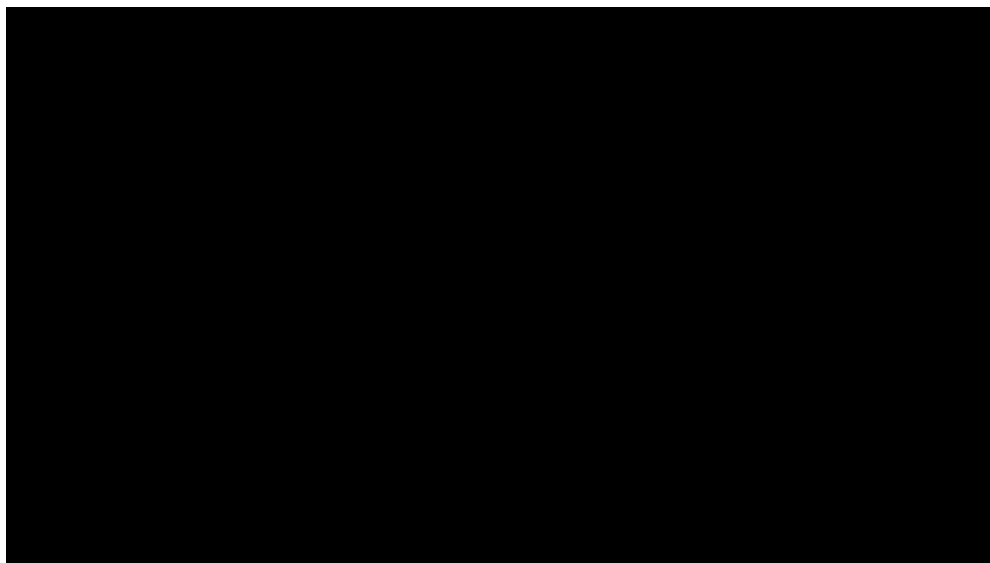
A Few RL Highlights

2013 - DQN (Deepmind)

2015 - DDPG (Deepmind)

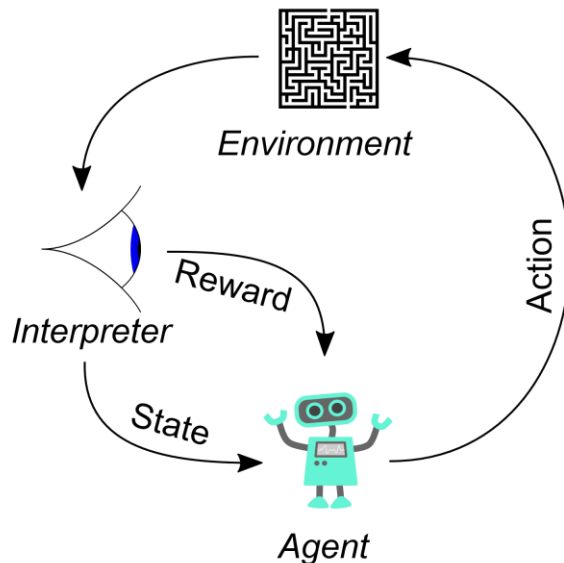
2017 - PPO (Berkley)

≥ 2017 – Real Robots



“Extreme Parkour with Legged Robots”, Cheng, Shi, et al, 2023

Reinforcement Learning: learning what to do through **interaction** with the environment, guided by **some rewards signal** that defines how much optimal is the chosen action



Reinforcement Learning: learning what to do through **interaction** with the environment, guided by **some rewards signal** that defines how much optimal is the chosen action

Supervised Learning

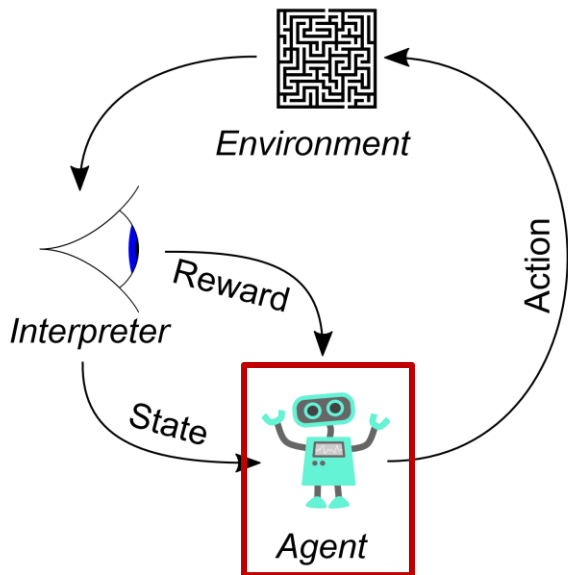
- learning from training set, which is **labeled from experienced person**
- goal: imitating the training set, with aim of generalize beyond it

Unsupervised Learning

- learning structure in **unlabeled data**
- goal: discover structure/representations

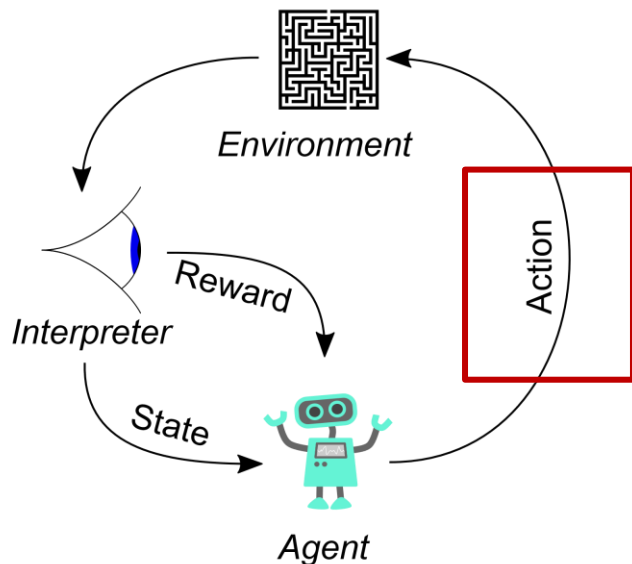
- Motivation
- **Problem Definition**
- Tabular Methods
- Deep Learning

MDPs formalize the sequential decision making problem, where the action influence not only the next reward, but also future signals



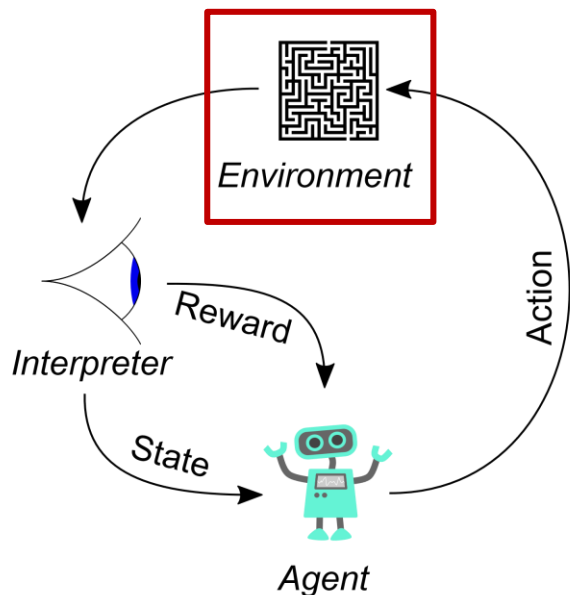
- **Agent:** Robots

MDPs formalize the sequential decision making problem, where the action influence not only the next reward, but also future signals



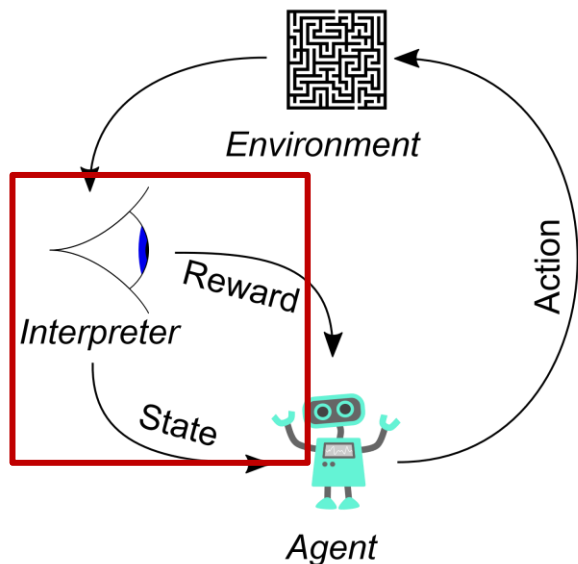
- **Agent:** Robots
- **Action:** Motor desired position

MDPs formalize the sequential decision making problem, where the action influence not only the next reward, but also future signals

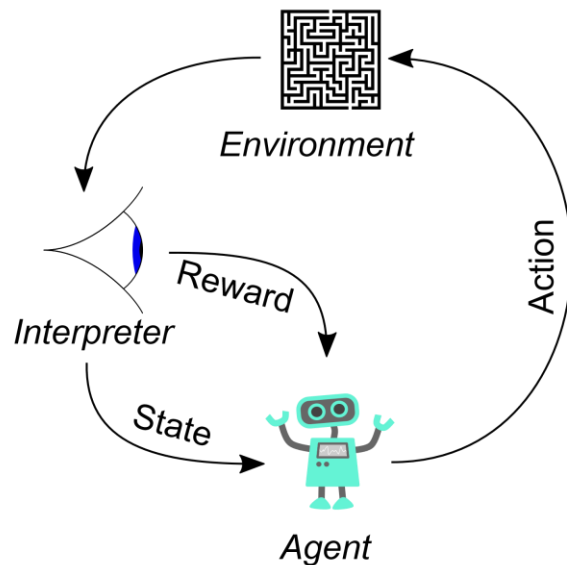


- **Agent:** Robots
- **Action:** Motor desired position
- **Environment:** Real world or simulators

MDPs formalize the sequential decision making problem, where the action influence not only the next reward, but also future signals



- **Agent:** Robots
 - **Action:** Motor desired position
- **Environment:** Real world or simulators
 - **New State:** Sensors information
- **Rewards:** Signal on how good the action was



In RL we maximize, not minimize.
This is only for historical reasons

$$r^{ocp} = ||q_d - q^{ref}||$$
$$r^{rl} = -||q_d - q^{ref}||$$



Some terminology

- Set of State S
- Set of Action A
- Transition Function $p(s_{k+1} | s_k, a_k)$
- Reward function $r(s_k, a_k, s_{k+1})$
- Sum of future rewards

$$R_k = r_k + r_{k+1} + r_{k+2} + r_{k+3} + \dots$$

- RL goal $\rightarrow \max_{\pi} E_{\pi} [\sum_{k=0}^{\infty} r_k]$

Some terminology

- Discount factor γ : “how much we should consider the future”

$$r_k \leftarrow \gamma r_{k+1}$$

$$r_k \leftarrow \gamma^2 r_{k+2}$$



$$R_k = r_k + \gamma r_{k+1} + \gamma^2 r_{k+2} + \gamma^3 r_{k+3} + \dots$$

- Set of State S
- Set of Action A
- Transition Function $p(s_{k+1} \mid s_k, a_k)$
- Reward function $r(s_k, a_k, s_{k+1})$
- Sum of future rewards

$$R_k = r_k + r_{k+1} + r_{k+2} + r_{k+3} + \dots$$

- RL goal $\rightarrow \max_{\pi} E_{\pi} [\sum_{k=0}^{\infty} r_k]$

Some terminology

- Discount factor γ : “how much we should consider the future”

$$r_k \leftarrow \gamma r_{k+1}$$

$$r_k \leftarrow \gamma^2 r_{k+2}$$



$$R_k = r_k + \gamma r_{k+1} + \gamma^2 r_{k+2} + \gamma^3 r_{k+3} + \dots$$

How much we can look into the future?

$$\gamma = 0.90 \quad \gamma^{30} \approx 0.04$$

$$\gamma = 0.99 \quad \gamma^{30} \approx 0.73$$

- Set of State S
- Set of Action A
- Transition Function $p(s_{k+1} \mid s_k, a_k)$
- Reward function $r(s_k, a_k, s_{k+1})$
- Sum of future rewards

$$R_k = r_k + r_{k+1} + r_{k+2} + r_{k+3} + \dots$$

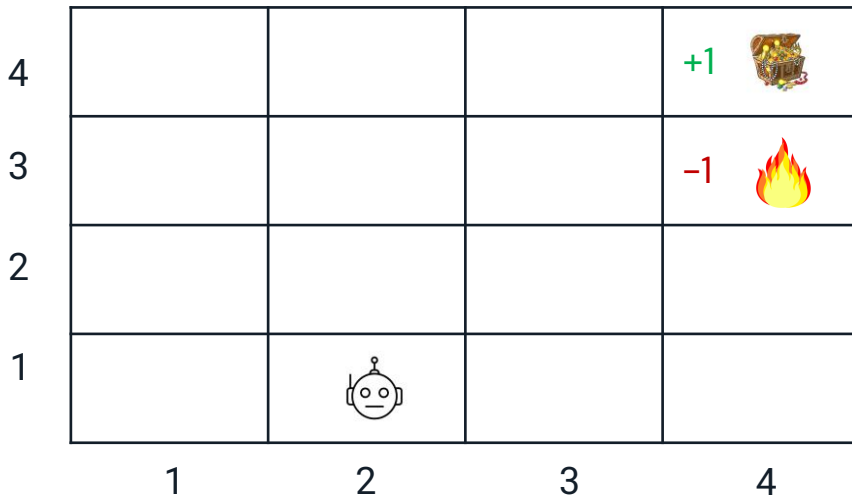
- RL goal $\rightarrow \max_{\pi} E_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_k \right]$

- Motivation
- Problem Definition
- **Tabular Methods**
 - Value Iteration
 - Q-iteration
- Deep Learning

Example: Gridworld

Some terminology

- Set of State S (**cell position**)
- Set of Action A (**up-down-left-right**)
- Transition Function $p(s_{k+1} \mid s_k, a_k)$
- Reward function $r(s_k, a_k, s_{k+1})$
- Discount factor γ
- **Goal** $\max_{\pi} E_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_k \right]$



Value Function (of a state s_t under a particular policy π): sum of discounted rewards when starting from state s_t and then following a particular policy π

$$V^\pi(s_t) = E\left[\sum_{k=0}^{\infty} \gamma^K r(s_{t+k}, a_{t+k}, s_{t+k+1})\right] \quad \text{given} \quad a_k = \pi(s_k)$$

Different policies have different **value functions**!



Value Function (of a state s_t under a particular policy π): sum of discounted rewards when starting from state s_t and then following a particular policy π

$$V^\pi(s_t) = E\left[\sum_{k=0}^{\infty} \gamma^K r(s_{t+k}, a_{t+k}, s_{t+k+1})\right] \quad \text{given} \quad a_k = \pi(s_k)$$

Optimal Value Function: sum of discounted rewards when starting from state s_t and then following **the optimal policy** π^*

$$V^*(s_t) = E\left[\sum_{k=0}^{\infty} \gamma^K r(s_{t+k}, a_{t+k}, s_{t+k+1})\right] \quad \text{given} \quad a_k = \pi^*(s_k)$$

Unknown! How to find it?



Example: Gridworld

$$V^*(4, 4) = 1$$

$$V^*(3, 4) = 0 + \gamma V^*(4, 4)$$

$$V^*(2, 4) = 0 + \gamma V^*(3, 4)$$

.....

What if each action is not deterministic?

$$V^*(4, 4) = 1$$

$$V^*(3, 4) = 0 + 0.8\gamma V^*(4, 4) + 0.1\gamma V^*(3, 3) + 0.1\gamma V^*(2, 4)$$

.....

$$V^\pi(s_t) = E\left[\sum_{k=0}^{\infty} \gamma^k r(s_{t+k}, a_{t+k}, s_{t+k+1})\right]$$

4				+1 
3				-1 
2				
1				
	1	2	3	4

Initialize all $V(s)$ to zero

For t iteration, until converge:

For each state s :

$$V_{t+1}(s) \leftarrow \max_a \sum_{s'} p(s'|a, s)(r + \gamma V_t(s'))$$

Output:

$$\pi(s) = \operatorname{argmax}_a \sum_{s'} p(s'|a, s)(r + \gamma V(s'))$$

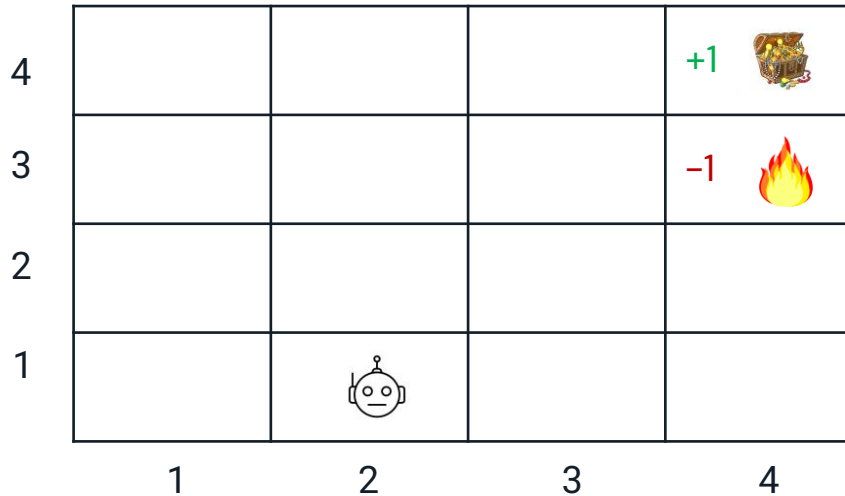


In every state, evaluate all the actions
and pick the best!

This is proven to converge after some steps

Example: Gridworld

From $t = 0$ to $t=100$

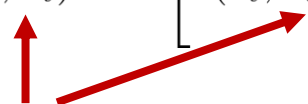


<https://colab.research.google.com/drive/1eUzr4PxFKQZ6eePojQu6mGk4xEWsT66e?usp=sharing>

Value Function (of a state s_t under a particular policy π): sum of discounted rewards when starting from state s_t and then following a particular policy π

$$V^\pi(s_t) = E\left[\sum_{k=0}^{\infty} \gamma^k r(s_{t+k}, a_{t+k}, s_{t+k+1})\right] \quad \text{given} \quad a_k = \pi(s_k)$$

Q-Function (of a state-action pair (s_t, a_t) , under a particular policy π): sum of discounted rewards when starting from state s_t , **taking action a_t** , and then following policy π .

$$Q^\pi(s_t, a_t) = E\left[r(s_t, a_t, s_{t+1}) + \sum_{k=1}^{\infty} \gamma^k r(s_{t+k}, a_{t+k}, s_{t+k+1})\right] \quad \text{given} \quad a_{k+1} = \pi(s_{k+1})$$


The first action is like a free variable – not chosen by the policy

Value Function (of a state s_t under a particular policy π): sum of discounted rewards when starting from state s_t and then following a particular policy π

$$V^\pi(s_t) = E\left[\sum_{k=0}^{\infty} \gamma^k r(s_{t+k}, a_{t+k}, s_{t+k+1})\right] \quad \text{given} \quad a_k = \pi(s_k)$$

Q-Function (of a state-action pair (s_t, a_t) , under a particular policy π): sum of discounted rewards when starting from state s_t , **taking action a_t** , and then following policy π .

$$Q^\pi(s_t, a_t) = E\left[r(s_t, a_t, s_{t+1}) + \sum_{k=1}^{\infty} \gamma^k r(s_{t+k}, a_{t+k}, s_{t+k+1})\right] \quad \text{given} \quad a_{k+1} = \pi(s_{k+1})$$

$$Q^\pi(s_t, a_t) = E[r(s_t, a_t, s_{t+1}) + \gamma V^\pi(s_{t+1})]$$

Initialize all $Q(s, a)$ to zero

For t iteration, until converge:

For each state s :

$$Q_{t+1}(s, a) \leftarrow \sum_{s'} p(s'|a, s)(r + \max_a \gamma Q(s', a'))$$

Output:

$$\pi(s) = \operatorname{argmax}_a Q(a, s)$$

Instead, in Value-Iteration, we have

$$\pi(s) = \operatorname{argmax}_a \sum_{s'} p(s'|a, s)(r + \gamma V(s'))$$

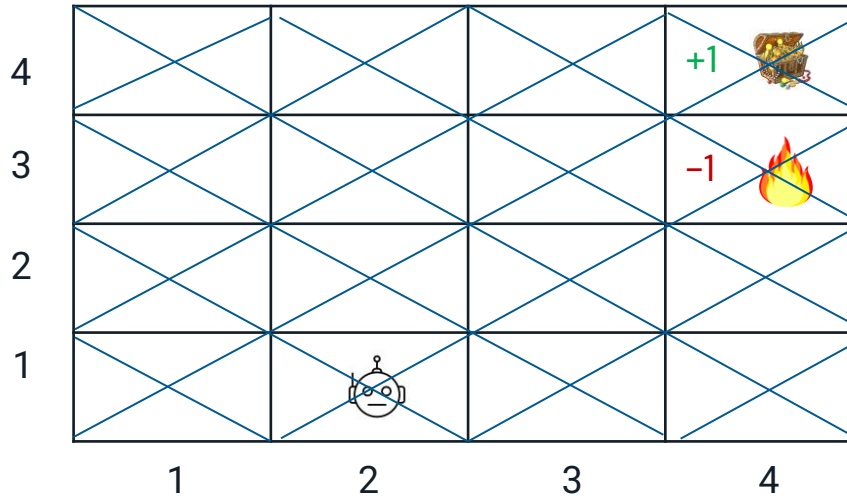


- In every state, evaluate all the actions and pick the best!
- **You need the dynamics of the system to look one step ahead!**

This is proven to converge after some steps

Example: Gridworld

From $t = 0$ to $t=100$



<https://colab.research.google.com/drive/1NebH13kYiUXbVtZiWp6icmKIMhcc1O37?usp=sharing>

Initialize all $Q(s, a)$ to zero

For t iteration, until converge:

For each state s :

$$Q_{t+1}(s, a) \leftarrow \sum_{s'} p(s'|a, s)(r + \max_a \gamma Q(s', a'))$$

Output:

$$\pi(s) = \operatorname{argmax}_a Q(a, s)$$

Initialize all $Q(s, a)$ to zero

For t iteration, until converge:

For each state s :

$$Q_{t+1}(s, a) \leftarrow \sum_{s'} p(s'|a, s)(r + \max_a \gamma Q(s', a'))$$

Output:

$$\pi(s) = \operatorname{argmax}_a Q(a, s)$$

Instead, in Value-Iteration, we have

$$\pi(s) = \operatorname{argmax}_a \sum_{s'} p(s'|a, s)(r + \gamma V(s'))$$

This is proven to converge after some steps

**Can we get rid of this dynamics as well?
Why?**

- In every state, evaluate all the actions and pick the best!
- **You need the dynamics of the system to look one step ahead!**

Initialize all $Q(s, a)$ to zero

For t iteration, until converge:

For each state s :

$$Q_{t+1}(s, a) \leftarrow \sum_{s'} p(s'|a, s) (r + \max_a \gamma Q(s', a'))$$

We can rewrite it as **an expectation**



$$Q_{t+1}(s, a) \leftarrow \mathbb{E}_{s' \sim p(s'|s, a)} \left[r + \gamma \max_{a'} Q_t(s', a') \right]$$

Idea: **Replacing expectation via sampling**

Initialize all $Q(s, a)$ to zero

For t iteration, until converge:

~~For each state s :~~

$$\cancel{Q_{t+1}(s, a) \leftarrow \mathbb{E}_{s' \sim p(s'|s, a)} \left[r + \gamma \max_{a'} Q_t(s', a') \right]}$$

Sample action $\mathbf{a}$, and go to state $\mathbf{s}'$, observe reward $\mathbf{r}$

$$target = r + \gamma \max_{a'} Q_t(s', a')$$

$$Q_{t+1}(s, a) \leftarrow (1 - \alpha)Q_t(s, a) + \alpha target$$

This is proven to converge after some steps - if you visit all the state “enough”

To visit all the states enough, **you need to samples actions accurately! (Exploration vs Exploitation)**

Initialize all $Q(s, a)$ to zero

For t iteration, until converge:

Flip a coin:

if head:

$$a = \operatorname{argmax}_a Q(s, a)$$

else:

$$a = \operatorname{choose_random_action}(s)$$

go to state $\mathbf{s}'$, observe reward $\mathbf{r}$

$$\text{target} = r + \gamma \max_{a'} Q_t(s', a')$$

$$Q_{t+1}(s, a) \leftarrow (1 - \alpha)Q_t(s, a) + \alpha \text{target}$$

Example: Pendulum



Pendulum

Observation:

angle pos $[-3.14, 3.14]$ rad

angle vel $[-2.0, 2.0]$ rad/s

Actions:

$[-10 \text{ Nm}, 10 \text{ Nm}]$

Continuous values to be discretized! E.g., using a 0.1 step

Observation:

angle pos = 62 values

angle vel = 40 values

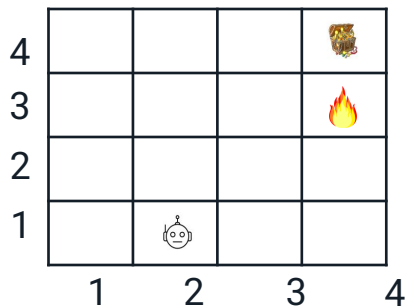
possible states = 2480

Actions:

torque = 200

<https://colab.research.google.com/drive/1r3k3lodXcosHg6e5W739k3gB1qqGJgcj?usp=sharing>

Curse of Dimensionality



Gridworld

Observations: 16 cells

Actions: 4 (up, down, left..)



Pendulum

Observation:

angle pos $[-3.14, 3.14]$ rad

angle vel $[-2.0, 2.0]$ rad/s

Actions:

$[-10 \text{ Nm}, 10 \text{ Nm}]$

Continuous values to be discretized!
E.g., using a 0.1 step

Observation:

angle pos = 62 values

angle vel = 40 values

possible states = 2480

Actions:

torque = 200



Double Pendulum

Observations:

angle 1 pos $[-3.14, 3.14]$ rad
angle 2 pos $[-3.14, 3.14]$ rad
angle vel 1 $[-2.0, 2.0]$ rad/s
angle vel 2 $[-2.0, 2.0]$ rad/s

Actions:

angle 1 torque $[-10, 10]$ Nm
angle 2 torque $[-10, 10]$ Nm

Continuous values to be discretized!
E.g., using a 0.1 step

Observation:

angle 1 pos = 62 values
angle 2 pos = 62 values
angle vel 1 = 40 values
angle vel 2 = 40 values

possible states = 6,150,400

Actions:

torque 1 = 200 values
torque 2 = 200 values

possible actions = 40000



- 43 joints to discretize in the observations
- action space (43 joints torques) between very high values

We cannot use tabular method!

- Motivation
- Problem Definition
- Tabular Methods
- **Deep Learning**
 - Deep Q-Learning
 - Continuous Control

- Tabular methods need to maintain a description of the value function/q function in memory.
Not enough RAM/Hard Disk to fit the entire world!

Row #	State vector	Left	Stay	Right
51242	[1, 6, 2, 6]	0.000000	0.000000	0.000000
51243	[1, 6, 2, 7]	0.000000	0.000000	0.000000
51244	[1, 6, 2, 8]	0.000000	0.000000	0.000000
51245	[1, 6, 3, 0]	7.843658	8.111676	9.612881
51246	[1, 6, 3, 1]	0.000000	0.000000	0.000000
51247	[1, 6, 3, 2]	0.000000	0.000000	0.000000
51248	[1, 6, 3, 3]	0.000000	0.000000	0.000000

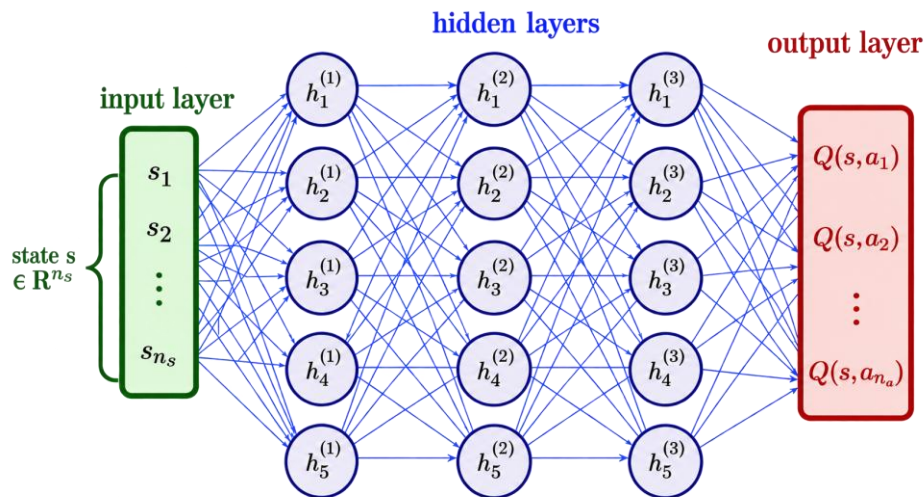
- Better solution: **Learning a nonlinear function instead of a table!**

- usually number of parameters to be learned <<
number of possible states)

$$Q_{\theta}(s, a) = \theta_1 \cos(s) + \theta_2 \sin(a)$$

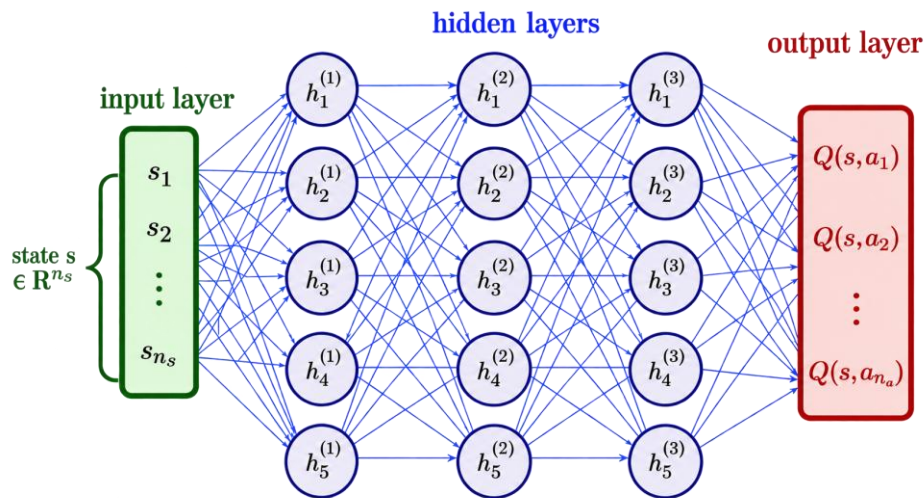
- functions can give us info of new states without visiting them (you can query the function at any state - **generalization!**)

- Which nonlinear function to use?



- Neural networks are **general function approximators**
- with enough neurons and suitable parameters, **they can approximate very complex nonlinear functions**

- Which nonlinear function to use?



- Our state for example

$$\mathbf{x} = \begin{bmatrix} s_1 \\ s_2 \end{bmatrix} = \begin{bmatrix} \text{joint positions} \\ \text{joint velocities} \end{bmatrix}$$

- arrows indicates parameters (weights) to be learned

$$\theta \leftarrow W, b$$

- h indicate neurons (mathematical operation)

$$z_i^{(1)} = \sum_j w_{ij}^{(1)} x_j + b_i^{(1)}$$

$$h_i^{(1)} = \tanh(z_i^{(1)})$$

Initialize all $Q(s, a)$ to zero

For t iteration, until converge:

Sample action $\mathbf{a}$, and go to state $\mathbf{s}'$, observe reward $\mathbf{r}$

$$target = r + \gamma \max_{a'} Q_t(s', a')$$

$$Q_{t+1}(s, a) \leftarrow (1 - \alpha) Q_t(s, a) + \alpha target$$

$$\theta_{k+1} \leftarrow \theta_k - \alpha \nabla_{\theta} \left[\frac{1}{2} (Q_{\theta}(s, a) - target(s'))^2 \right]_{\theta=\theta_k}$$



Neural Network learning framework (pytorch, jax) compute this update by themselves

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

With probability ε select a random action a_t

otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

End For

End For

a replay buffer is used to store transition

downsampling image

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

With probability ε select a random action a_t

otherwise select $a_t = \arg\max_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

Every C steps reset $\hat{Q} = Q$

End For

End For



exploration

Algorithm 1: deep Q-learning with experience replay.

Initialize replay memory D to capacity N

Initialize action-value function Q with random weights θ

Initialize target action-value function $\hat{Q}$ with weights $\theta^- = \theta$

For episode = 1, M **do**

Initialize sequence $s_1 = \{x_1\}$ and preprocessed sequence $\phi_1 = \phi(s_1)$

For $t = 1, T$ **do**

With probability ε select a random action a_t

otherwise select $a_t = \operatorname{argmax}_a Q(\phi(s_t), a; \theta)$

Execute action a_t in emulator and observe reward r_t and image x_{t+1}

Set $s_{t+1} = s_t, a_t, x_{t+1}$ and preprocess $\phi_{t+1} = \phi(s_{t+1})$

Store transition $(\phi_t, a_t, r_t, \phi_{t+1})$ in D

Sample random minibatch of transitions $(\phi_j, a_j, r_j, \phi_{j+1})$ from D

Set $y_j = \begin{cases} r_j & \text{if episode terminates at step } j+1 \\ r_j + \gamma \max_{a'} \hat{Q}(\phi_{j+1}, a'; \theta^-) & \text{otherwise} \end{cases}$

Perform a gradient descent step on $(y_j - Q(\phi_j, a_j; \theta))^2$ with respect to the network parameters θ

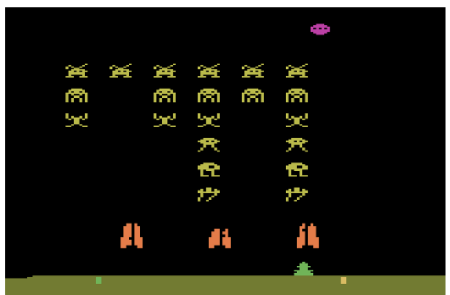
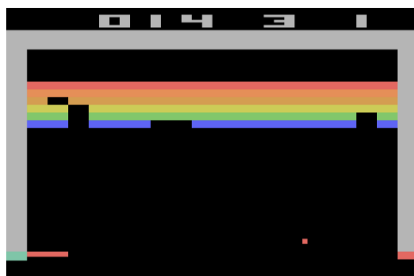
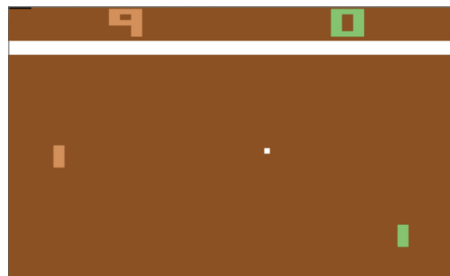
Every C steps reset $\hat{Q} = Q$

End For

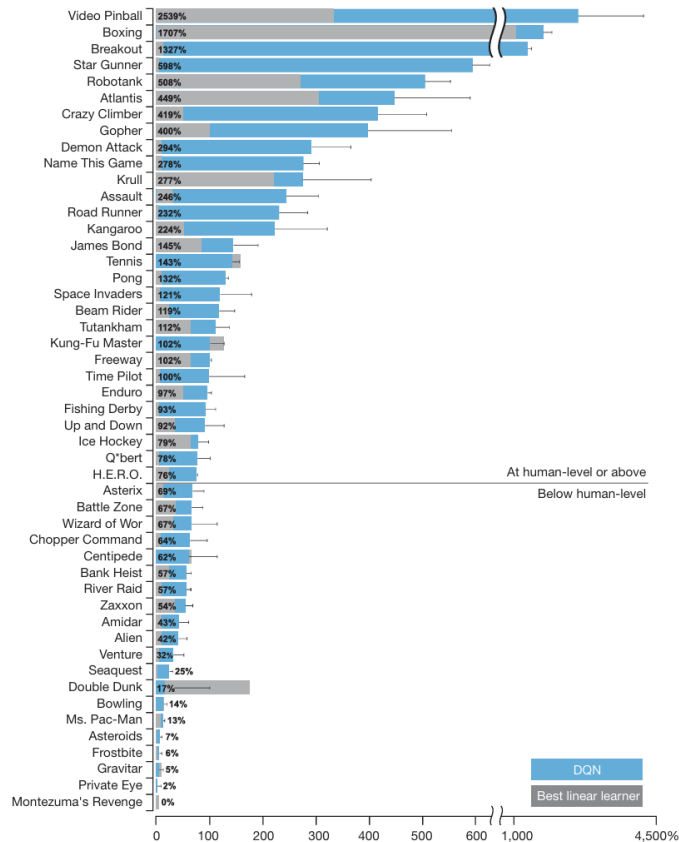
End For

Parameters update done on multiple transition at once = more stable training

Deep Q-Learning



“Playing Atari with Deep Reinforcement Learning”, Mnih et al, 2013





Double Pendulum

Observations:

angle 1 pos $[-3.14, 3.14]$ rad
angle 2 pos $[-3.14, 3.14]$ rad
angle vel 1 $[-2.0, 2.0]$ rad/s
angle vel 2 $[-2.0, 2.0]$ rad/s

Actions:

angle 1 torque $[-10, 10]$ Nm
angle 2 torque $[-10, 10]$ Nm

Continuous values to be discretized!
E.g., using a 0.1 step

Observation:

angle 1 pos = 62 values
angle 2 pos = 62 values
angle vel 1 = 40 values
angle vel 2 = 40 values

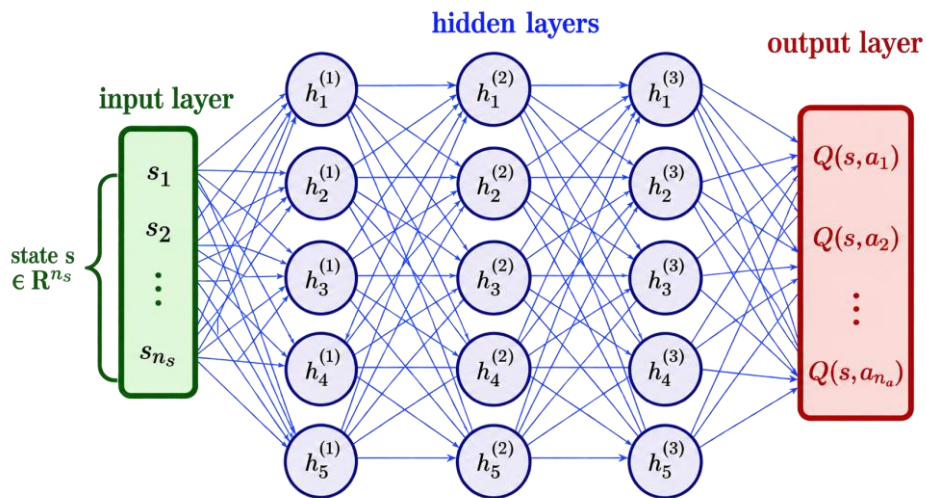
possible states = 6,150,400

Actions:

torque 1 = 200 values
torque 2 = 200 values

possible actions = 40000

Deep Q-Learning Problems



- Observations of NN are continuous – **no need of state discretization!**
- The number of outputs of $Q(s, a)$ can be quite big!
- Double Pendulum **possible actions = 40000**
- Compute the **argmax** of all the possible actions is still too much

- Motivation
- Markov Decision Process
- Tabular Methods
 - Value Iteration
 - Q-Iteration
- **Deep Learning**
 - **Deep-Q-Learning**
 - **Policy Optimization**

- DQN choose the optimal action as $a^* = \operatorname{argmax}_a Q(s, a)$
- What if we learn directly the policy, without any argmax?

$$a = \pi(s|\theta)$$

Specific neural network directly to learn this mapping. How?

- Starting point:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right]$$

How can we change this parameters to maximize it?

- First idea: simple gradient update

$$\theta \leftarrow \theta + \alpha \nabla_{\theta} J(\theta)$$

- How does it look like the gradient of the cost function? (**skipping a lot of details...**)

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left(\sum_{k=t}^T \gamma^{k-t} r_k \right) \right]$$

- **How to collect this gradient? What this gradient does?**

Vanilla Policy Gradient

```
1: Initialize policy parameters  $\theta$ 
2: for iteration = 1, 2, ... do
3:   Collect  $N$  trajectories by executing  $\pi_\theta$ 
4:   for each trajectory  $i = 1, \dots, N$  do
5:     for each timestep  $t = 0, \dots, T_i - 1$  do
6:       Compute the return:
7:         
$$R_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_k^{(i)}$$

8:     end for
9:   end for
10:  Compute the policy-gradient estimate:
11:    
$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta \left( a_t^{(i)} \mid s_t^{(i)} \right) R_t^{(i)}$$

12:  Update the policy:
13:    
$$\theta \leftarrow \theta + \alpha \hat{g}$$

14: end for
```

Vanilla Policy Gradient

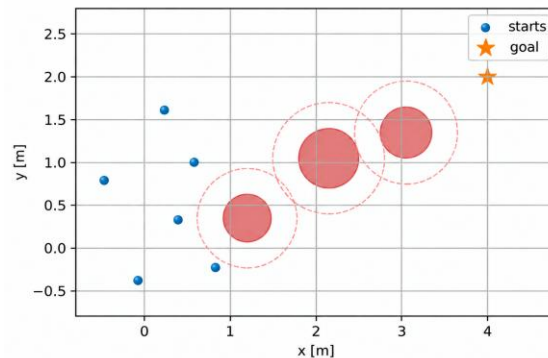
```
1: Initialize policy parameters  $\theta$ 
2: for iteration = 1, 2, ... do
3:   Collect  $N$  trajectories by executing  $\pi_\theta$ 
4:   for each trajectory  $i = 1, \dots, N$  do
5:     for each timestep  $t = 0, \dots, T_i - 1$  do
6:       Compute the return:
7:          $R_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_k^{(i)}$ 
8:     end for
9:   end for
10:  Compute the policy-gradient estimate:
11:     $\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta \left( a_t^{(i)} \mid s_t^{(i)} \right) R_t^{(i)}$ 
12:  Update the policy:
13:     $\theta \leftarrow \theta + \alpha \hat{g}$ 
14: end for
```

Random neural network
parameters

Vanilla Policy Gradient

- 1: Initialize policy parameters θ
- 2: **for** iteration = 1, 2, ... **do**
- 3: Collect N trajectories by executing π_θ
- 4: **for** each trajectory $i = 1, \dots, N$ **do**
- 5: **for** each timestep $t = 0, \dots, T_i - 1$ **do**
- 6: Compute the return:
- 7:
$$R_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_k^{(i)}$$
- 8: **end for**
- 9: **end for**
- 10: Compute the policy-gradient estimate:
- 11:
$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta \left(a_t^{(i)} \mid s_t^{(i)} \right) R_t^{(i)}$$
- 12: Update the policy:
- 13:
$$\theta \leftarrow \theta + \alpha \hat{g}$$
- 14: **end for**

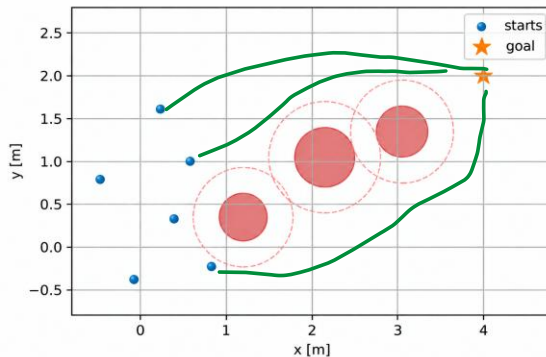
Initialize the robot in different initial state, and execute the policy, e.g.



Vanilla Policy Gradient

- 1: Initialize policy parameters θ
- 2: **for** iteration = 1, 2, ... **do**
- 3: Collect N trajectories by executing π_θ
- 4: **for** each trajectory $i = 1, \dots, N$ **do**
- 5: **for** each timestep $t = 0, \dots, T_i - 1$ **do**
- 6: Compute the return:
- 7:
$$R_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_k^{(i)}$$
- 8: **end for**
- 9: **end for**
- 10: Compute the policy-gradient estimate:
- 11:
$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta \left(a_t^{(i)} \mid s_t^{(i)} \right) R_t^{(i)}$$
- 12: Update the policy:
- 13:
$$\theta \leftarrow \theta + \alpha \hat{g}$$
- 14: **end for**

Each trajectory has different returns



Vanilla Policy Gradient

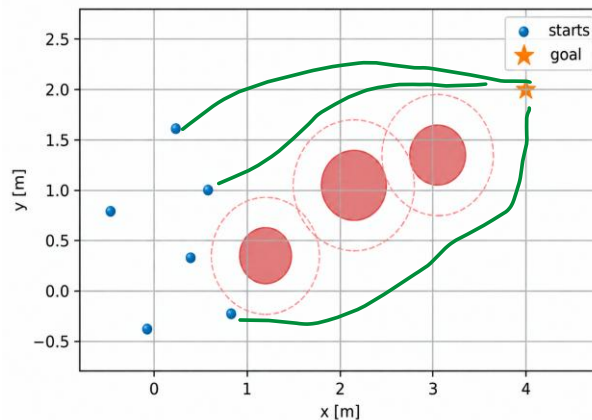
```
1: Initialize policy parameters  $\theta$ 
2: for iteration = 1, 2, ... do
3:   Collect  $N$  trajectories by executing  $\pi_\theta$ 
4:   for each trajectory  $i = 1, \dots, N$  do
5:     for each timestep  $t = 0, \dots, T_i - 1$  do
6:       Compute the return:
7:          $R_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_k^{(i)}$ 
8:     end for
9:   end for
10:  Compute the policy-gradient estimate:
11:     $\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta \left( a_t^{(i)} \mid s_t^{(i)} \right) R_t^{(i)}$ 
12:  Update the policy:
13:     $\theta \leftarrow \theta + \alpha \hat{g}$ 
14: end for
```

← The gradient move the policy in the direction of better returns

- To obtain PPO, the most used algorithm nowadays, we need two other modifications

1. Remember the gradient update

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta} \left(a_t^{(i)} \mid s_t^{(i)} \right) R_t^{(i)}$$



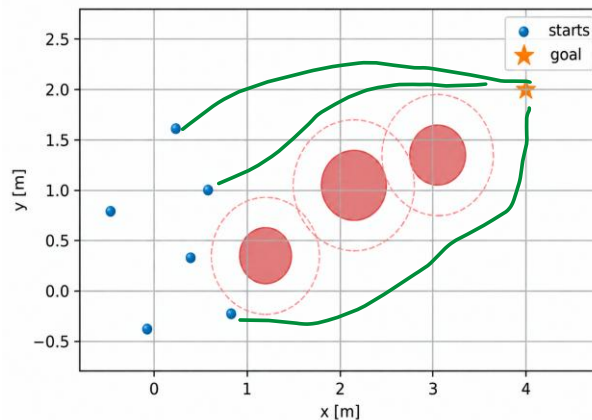
The return R_t can have **high variance (bad for learning signals)**: trajectories starting from different states may obtain very different returns.

A low return does not necessarily mean that the action was poor—it may simply reflect **that the agent started farther from the goal**.

- To obtain PPO, the most used algorithm nowadays, we need two other modifications

1. Remember the gradient update

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta} \left(a_t^{(i)} \mid s_t^{(i)} \right) R_t^{(i)}$$

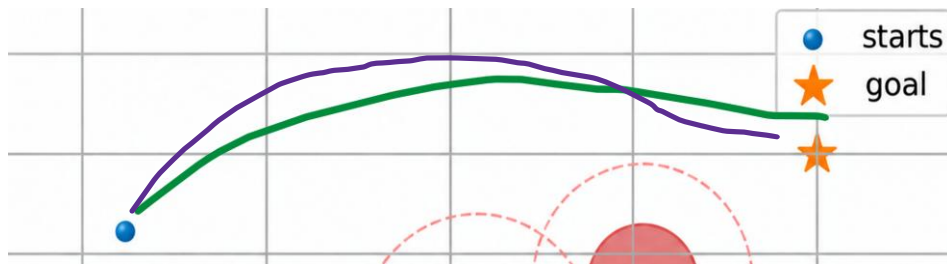
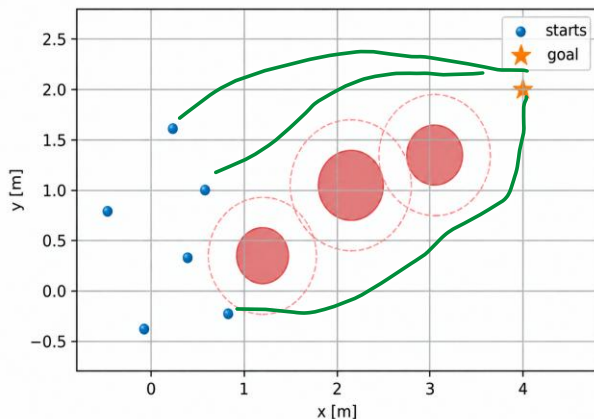


The return R_t can have **high variance (bad for learning signals)**: trajectories starting from different states may obtain very different returns.

Idea: Replace R_t with the **advantage function** $\hat{A}_t = R_t - V_{\phi}(s_t)$

Idea: Replace R_t with the **advantage function**

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta} \left(a_t^{(i)} \mid s_t^{(i)} \right) A_t^{(i)}$$

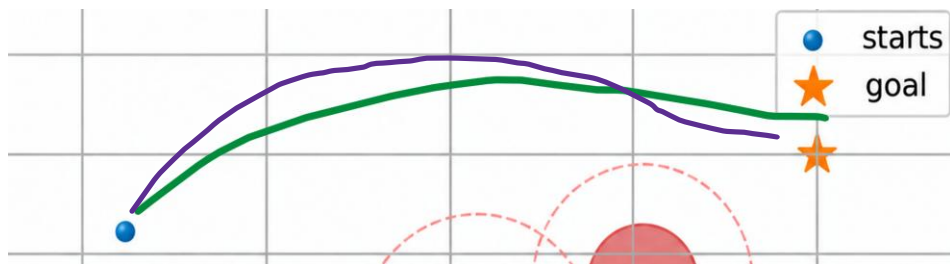


This **trajectory** is generated over some exploration noise. We should evaluate it against **the expected return from the same state under the current policy**, to see if the additional action helped!

Idea: Replace R_t with the **advantage function**

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta} \left(a_t^{(i)} \mid s_t^{(i)} \right) A_t^{(i)}$$

Expected return of policy – this can be learned with **neural network** as well!



This **trajectory** is generated over some exploration noise. We should evaluate it against **the expected return from the same state under the current policy**, to see if the additional action helped!

$$\hat{A}_t = R_t - V_{\phi}(s_t)$$



> 0: the new trajectory is better
< 0: no improvements!



Vanilla Policy Gradient

```

1: Initialize policy parameters  $\theta$ 
2: for iteration = 1, 2, ... do
3:   Collect  $N$  trajectories by executing  $\pi_\theta$ 
4:   for each trajectory  $i = 1, \dots, N$  do
5:     for each timestep  $t = 0, \dots, T_i - 1$  do
6:       Compute the return:
7:        $R_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_k^{(i)}$ 
8:     end for
9:   end for
10:  Compute the policy-gradient estimate:
11:   $\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta \left( a_t^{(i)} \mid s_t^{(i)} \right) R_t^{(i)}$ 
12:  Update the policy:
13:   $\theta \leftarrow \theta + \alpha \hat{g}$ 
14: end for
  
```

Policy Gradient with a Learned Baseline

```

1: Initialize policy parameters  $\theta$  and critic parameters  $\phi$ 
2: for iteration = 1, 2, ... do
3:   Collect  $N$  trajectories by executing  $\pi_\theta$ 
4:   for each trajectory  $i = 1, \dots, N$  do
5:     for each timestep  $t = 0, \dots, T_i - 1$  do
6:       Compute the return:
7:        $R_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_k^{(i)}$ 
8:       Compute the advantage estimate:
9:        $\hat{A}_t^{(i)} = R_t^{(i)} - V_\phi \left( s_t^{(i)} \right)$ 
10:    end for
11:  end for
12:  Update the critic by minimizing:
13:   $\mathcal{L}_{\text{critic}} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \left( V_\phi \left( s_t^{(i)} \right) - R_t^{(i)} \right)^2$ 
14:  Compute the policy-gradient estimate:
15:   $\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_\theta \log \pi_\theta \left( a_t^{(i)} \mid s_t^{(i)} \right) \hat{A}_t^{(i)}$ 
16:  Update the policy:
17:   $\theta \leftarrow \theta + \alpha \hat{g}$ 
18: end for
  
```

Link Vanilla Policy Gradient:

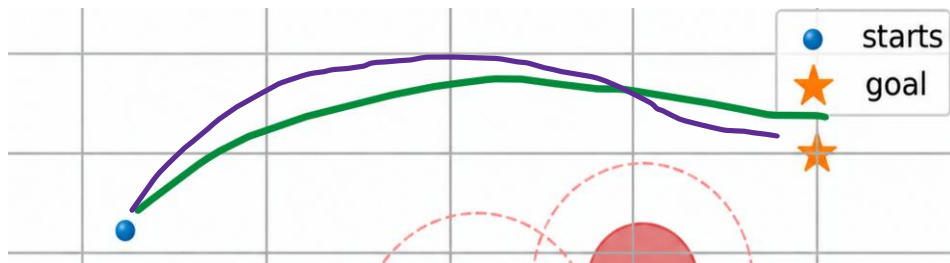
<https://colab.research.google.com/drive/1rjT50SU74CMxYTywoB-FVA6y5VnWrJ-q?usp=sharing>

Link A2C: <https://colab.research.google.com/drive/1x7JUmqOQ3nP-sjZZc3ZMsXejcDGa3kdKz?usp=sharing>

Idea: Replace R_t with the **advantage function**

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta} \left(a_t^{(i)} \mid s_t^{(i)} \right) A_t^{(i)}$$

Expected return of policy – this can be learned with **neural network** as well!



This **trajectory** is generated over some exploration noise. We should evaluate it against **the expected return from the same state under the current policy**, to see if the additional action helped!

$$\hat{A}_t = R_t - V_{\phi}(s_t)$$



> 0: the new trajectory is better
< 0: no improvements!



- To obtain PPO, the most used algorithm nowadays, we need two other modifications

2. After each learning iteration, the policy **can change too much!**

Compute the policy-gradient estimate:

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta} \left(a_t^{(i)} \mid s_t^{(i)} \right) \hat{A}_t^{(i)}$$

Update the policy:

$$\theta \leftarrow \theta + \alpha \hat{g}$$

α too small, we do not learn anything

α too big, we change too much...



- To obtain PPO, the most used algorithm nowadays, we need two other modifications

2. After each learning iteration, the policy **can change too much!**

Compute the policy-gradient estimate:

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta} \left(a_t^{(i)} \mid s_t^{(i)} \right) \hat{A}_t^{(i)}$$

Update the policy:

$$\theta \leftarrow \theta + \alpha \hat{g}$$

PPO clips the update,
constraining how much the policy
can change wrt the previous one

α too small, we do not learn anything

α too big, we change too much...

- To obtain PPO, the most used algorithm nowadays, we need two other modifications

3. Introduce a surrogate objective to optimize

$$L(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \underbrace{\frac{\pi_{\theta}(a_t^{(i)} | s_t^{(i)})}{\pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)})}}_{r_t^{(i)}(\theta)} \hat{A}_t^{(i)}$$

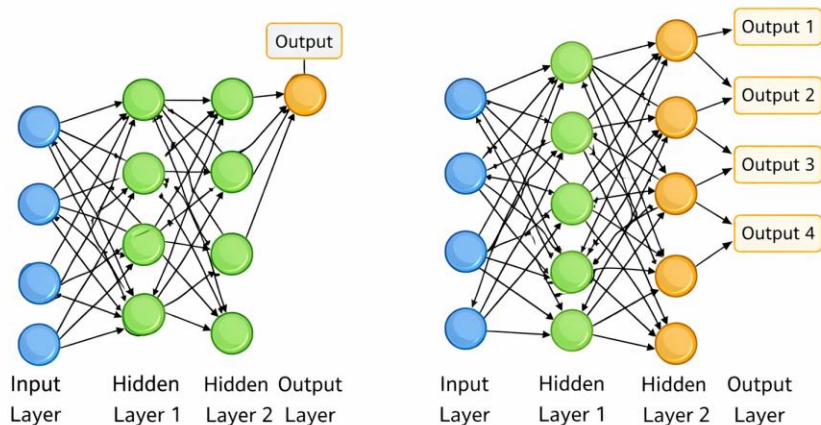
Now

$$\hat{g}(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} r_t^{(i)}(\theta) \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) \hat{A}_t^{(i)}$$

Before

$$\hat{g} = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) A_t^{(i)}$$

The most used RL algorithm nowadays is PPO (2017)



Critic: $V(s)$

Actor: $\pi(s)$

Only used in simulation! Used in simulation and deploy

Actor: propose an action

Critic: evaluate that action

 Proximal Policy Optimization (PPO)

- 1: Initialize policy parameters θ and critic parameters ϕ
- 2: **for** iteration = 1, 2, ... **do**
- 3: Set old policy parameters $\theta_{\text{old}} \leftarrow \theta$
- 4: Collect N trajectories by executing $\pi_{\theta_{\text{old}}}$
- 5: **for** each trajectory $i = 1, \dots, N$ **do**
- 6: **for** each timestep $t = 0, \dots, T_i - 1$ **do**
- 7: Compute the return:

$$R_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_k^{(i)}$$

- 8: Compute the advantage estimate:

$$\hat{A}_t^{(i)} = R_t^{(i)} - V_\phi(s_t^{(i)})$$

- 9: **end for**
- 10: **end for**
- 11: Update the critic by minimizing:

$$\mathcal{L}_{\text{critic}}(\phi) = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \left(V_\phi(s_t^{(i)}) - R_t^{(i)} \right)^2$$

- 12: Compute the probability ratio:

$$\rho_t^{(i)}(\theta) = \frac{\pi_\theta(a_t^{(i)} | s_t^{(i)})}{\pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)})}$$

- 13: Update the policy by maximizing:

$$\mathcal{L}_{\text{PPO}}(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \min \left(\rho_t^{(i)}(\theta) \hat{A}_t^{(i)}, \text{clip} \left(\rho_t^{(i)}(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t^{(i)} \right)$$

- 14: **end for**
-

 Proximal Policy Optimization (PPO)

- 1: Initialize policy parameters θ and critic parameters ϕ
- 2: **for** iteration = 1, 2, ... **do**
- 3: Set old policy parameters $\theta_{\text{old}} \leftarrow \theta$
- 4: Collect N trajectories by executing $\pi_{\theta_{\text{old}}}$
- 5: **for** each trajectory $i = 1, \dots, N$ **do**
- 6: **for** each timestep $t = 0, \dots, T_i - 1$ **do**
- 7: Compute the return:

$$R_t^{(i)} = \sum_{k=t}^{T_i-1} \gamma^{k-t} r_k^{(i)}$$

- 8: Compute the advantage estimate:

$$\hat{A}_t^{(i)} = R_t^{(i)} - V_\phi(s_t^{(i)})$$

- 9: **end for**
- 10: **end for**
- 11: Update the critic by minimizing:

$$\mathcal{L}_{\text{critic}}(\phi) = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \left(V_\phi(s_t^{(i)}) - R_t^{(i)} \right)^2$$

- 12: Compute the probability ratio:

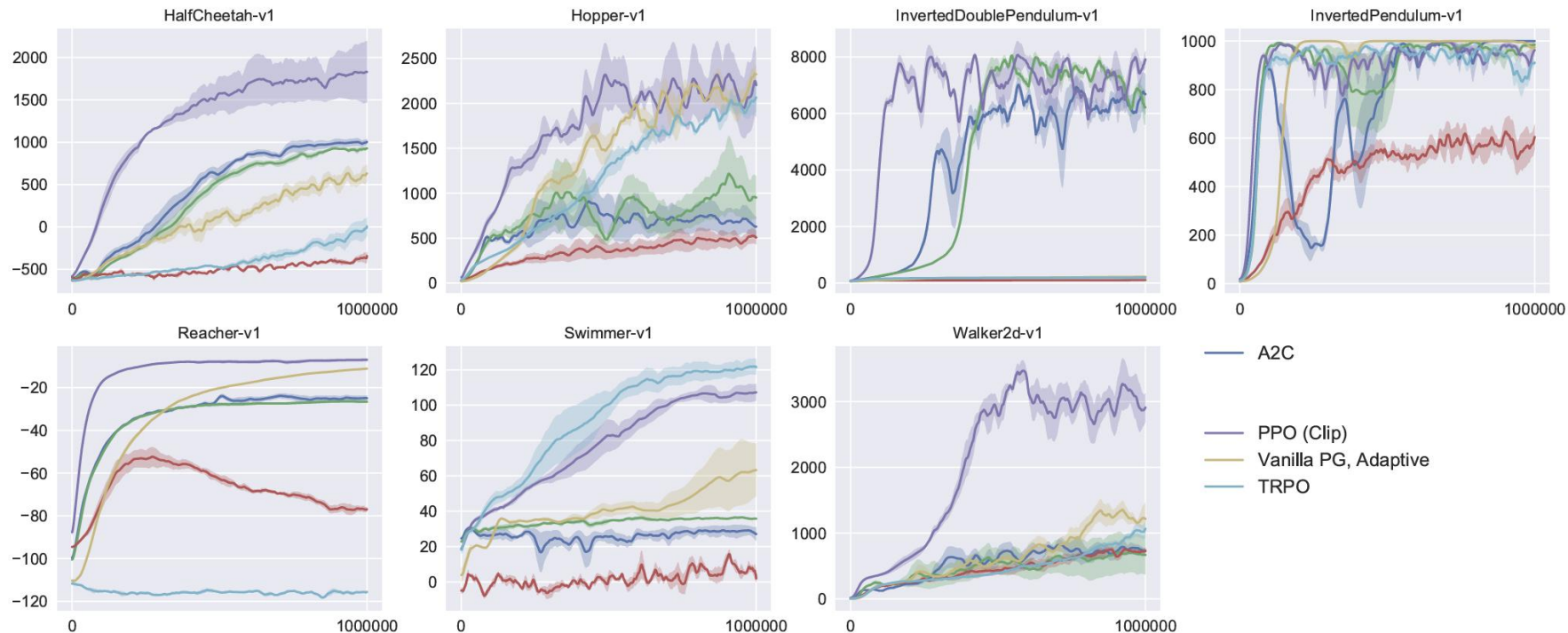
$$\rho_t^{(i)}(\theta) = \frac{\pi_\theta(a_t^{(i)} | s_t^{(i)})}{\pi_{\theta_{\text{old}}}(a_t^{(i)} | s_t^{(i)})}$$

- 13: Update the policy by maximizing:

$$\mathcal{L}_{\text{PPO}}(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T_i-1} \min \left(\rho_t^{(i)}(\theta) \hat{A}_t^{(i)}, \text{clip} \left(\rho_t^{(i)}(\theta), 1 - \epsilon, 1 + \epsilon \right) \hat{A}_t^{(i)} \right)$$

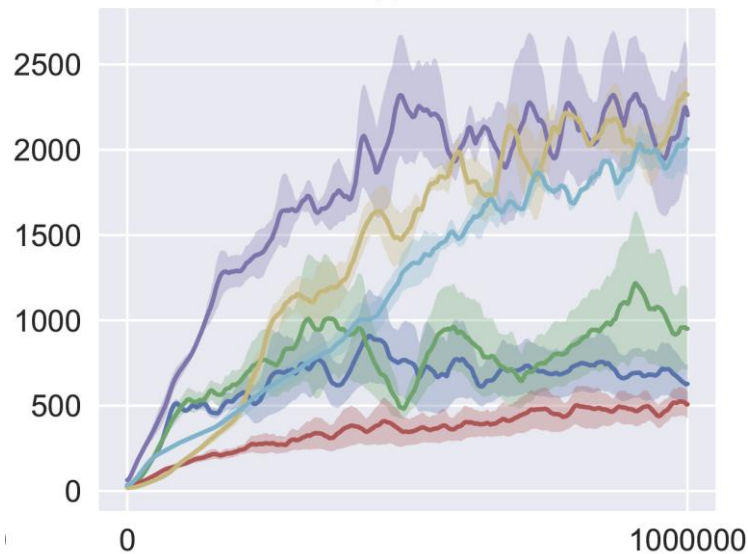
- 14: **end for**
-

PPO - Results



PPO - Results

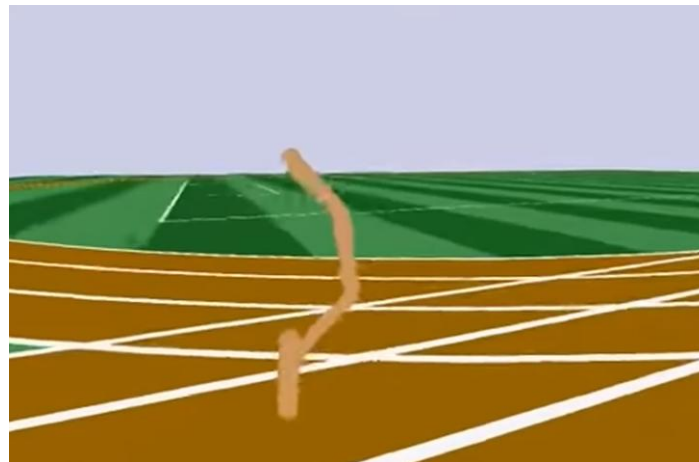
Hopper-v1



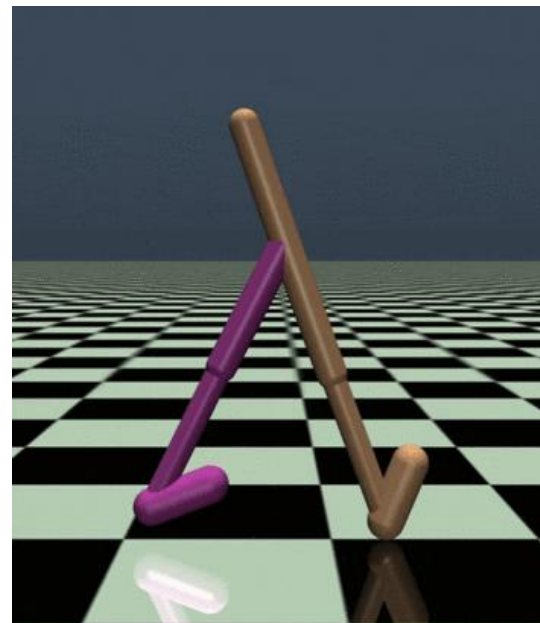
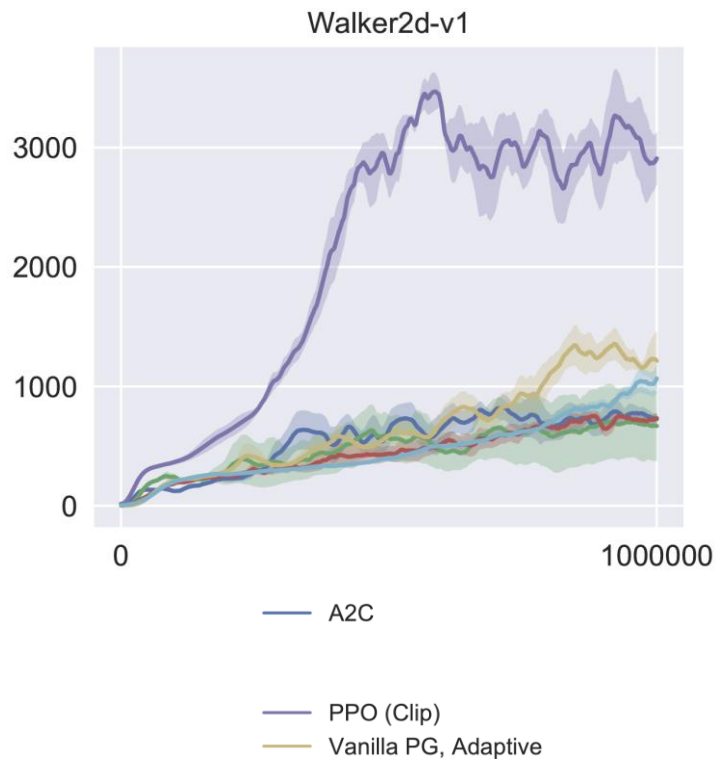
A2C

PPO (Clip)

Vanilla PG, Adaptive



PPO - Results



High-speed control and navigation for quadrupedal robots on complex and discrete terrain

Hyeongju Kim, Hyunsik Oh, Jeongsoo Park,
Yunho Kim, Donghoon Youm, Moonkyu Jung,
Minho Lee, Jemin Hwangbo*



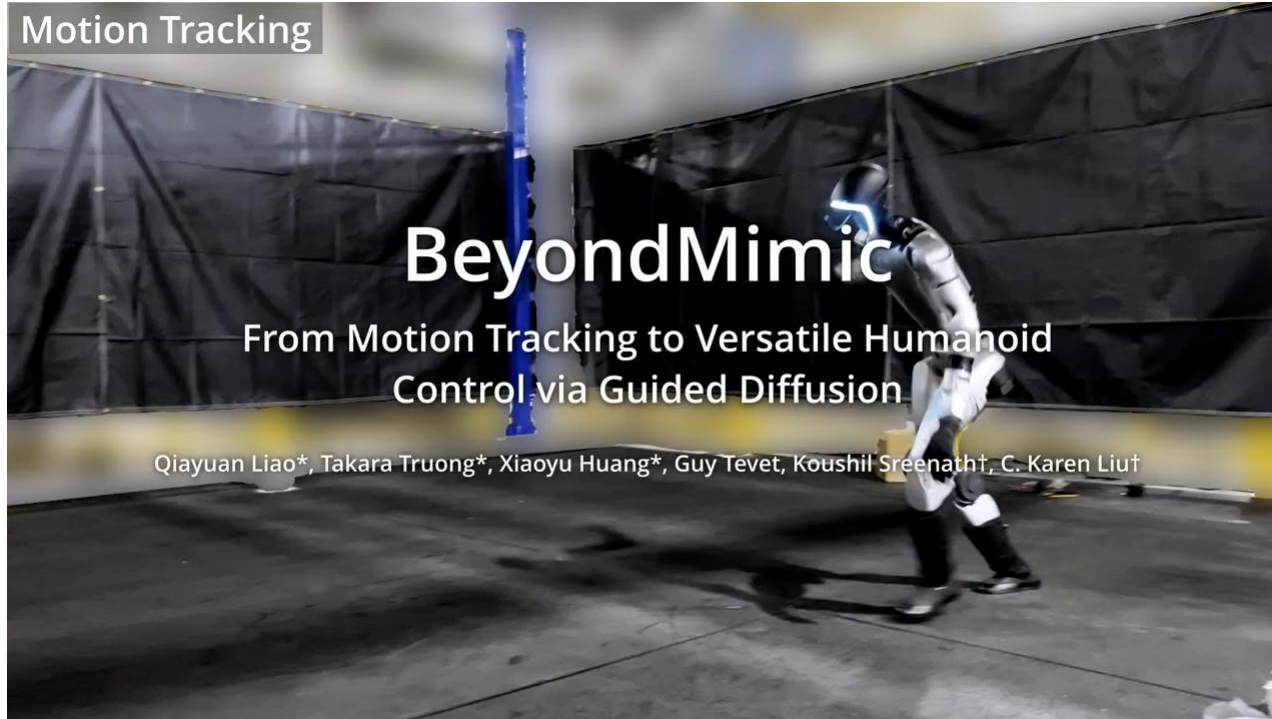
SAMSUNG

Motion Tracking

BeyondMimic

From Motion Tracking to Versatile Humanoid
Control via Guided Diffusion

Qiayuan Liao*, Takara Truong*, Xiaoyu Huang*, Guy Tevet, Koushil Sreenath†, C. Karen Liut





We introduce a novel hopping robot, designed as a single leg from a bipedal humanoid.



Body Design and Gait Generation of Chair-Type Asymmetrical Tripedal Low-rigidity Robot

Shintaro Inoue¹, Kento Kawaharazuka¹,
Kei Okada¹, and Masayuki Inaba¹
(¹The University of Tokyo)

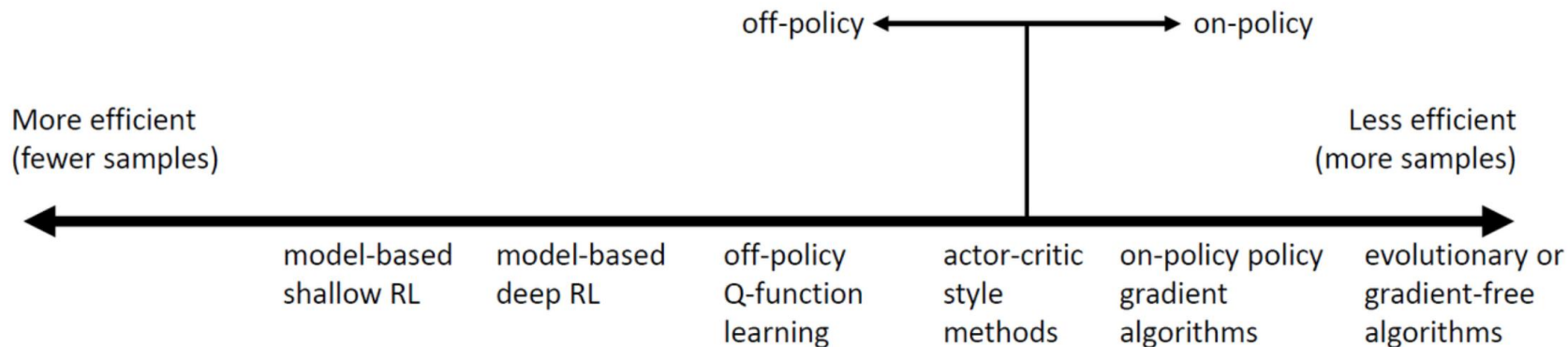
RoboSoft2024 – IEEE-RAS International Conference on Soft Robotics



“Fingers as Legs: Learning Self-Supported Locomotion and Manipulation with an Anthropomorphic Hand”,
Kazemipour et al, 2026

- **Vanilla Policy Gradient** can solve continuous actions problems...but it had **high variance**
- **A2C** introduced the concept of Advantage Estimation, to reduce it...but the policy update can **be too abrupt sometimes**
- **PPO** introduced an easy way to do clip of update (this concept was introduced a little bit earlier with TRPO, but more complex)...**but?**

Landscape of RL



The policy $\pi(s)$ can be improved based **on all the data in the past!**

General learning iteration:

```
D = {} # empty array
for i in range(0, num_iterations):
    state <- get_observation()
    action <- get_action(state)
    next_state <- simulate(action, state)
    r <- get_reward(action, state)
    D = D U (next_state, action, state)
    update_policy(D)
```

Even if $\pi(s)$ changes at every iteration, **old data remains valid!**

The policy $\pi(s)$ can be improved based **on the most recent data!**

General learning iteration:

```
D = {} # empty array  
for i in range(0, num_iterations):  
    state <- get_observation()  
    action <- get_action(state)  
    next_state <- simulate(action, state)  
    r <- get_reward(action, state)  
    D = (next_state, action, state)  
    update_policy(D)
```

When $\pi(s)$ changes at every iteration, **old data is discarded!**

Off-Policy

Pro

Sample-efficient – older sample are not discarded

Cons

High GPU VRAM – maintaining samples is memory consuming

On-policy

Pro

Low GPU VRAM – older sample are discarded

Cons

Not Sample-efficient – older sample are discarded – need to generate new data every time

Off-Policy

SAC

TD3

Fast-SAC

Fast-TD3

On-policy

A2C

TRPO

PPO

README Code of conduct Contributing MIT license

🔗 ✎ ☰

CI passing docs passing coverage 96.00% code style black

Stable Baselines3

Stable Baselines3 (SB3) is a set of reliable implementations of reinforcement learning algorithms in PyTorch. It is the next major version of [Stable Baselines](#).

You can read a detailed presentation of Stable Baselines3 in the [v1.0 blog post](#) or our [JMLR paper](#).

These algorithms will make it easier for the research community and industry to replicate, refine, and identify new ideas, and will create good baselines to build projects on top of. We expect these tools will be used as a base around which new ideas can be added, and as a tool for comparing a new approach against existing ones. We also hope that the simplicity of these tools will allow beginners to experiment with a more advanced toolset, without being buried in implementation details.



A lot of different algorithms implemented:

- DQN
- A2C
- PPO
- DDPG
- SAC

RSL-RL: A Learning Library for Robotics Research

Clemens Schwarke^{1,2}

Mayank Mittal^{1,2}

Nikita Rudin^{1,2,3}

David Hoeller^{1,2,3}

Marco Hutter¹

¹ETH Zürich, ²NVIDIA, ³Flexion Robotics

CSCHWARKE@ETHZ.CH

MITTALMA@ETHZ.CH

NIKITA@FLEXION.AI

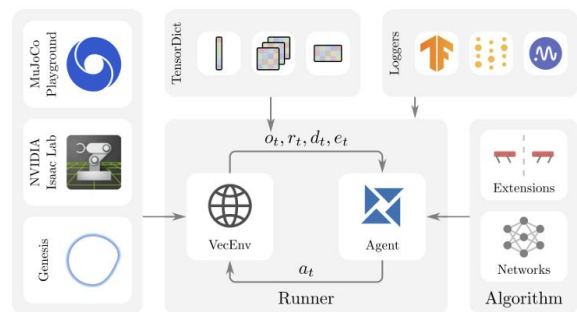
DAVID@FLEXION.AI

MAHUTTER@ETHZ.CH

Abstract

RSL-RL is an open-source Reinforcement Learning library tailored to the specific needs of the robotics community. Unlike broad general-purpose frameworks, its design philosophy prioritizes a compact and easily modifiable codebase, allowing researchers to adapt and extend algorithms with minimal overhead. The library focuses on algorithms most widely adopted in robotics, together with auxiliary techniques that address robotics-specific challenges. Optimized for GPU-only training, RSL-RL achieves high-throughput performance in large-scale simulation environments. Its effectiveness has been validated in both simulation benchmarks and in real-world robotic experiments, demonstrating its utility as a lightweight, extensible, and practical framework to develop learning-based robotic controllers. The library is open-sourced at: https://github.com/leggedrobotics/rsl_rl.

Keywords: Reinforcement Learning, Distillation, Robotics, PyTorch

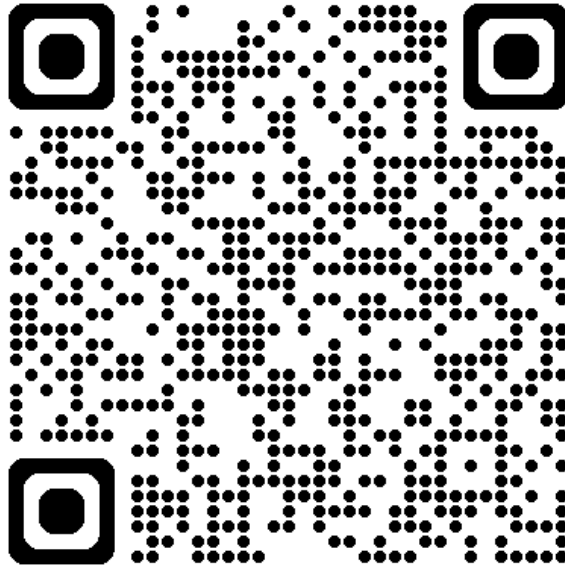


- Focused only on PPO
- Most used RL library for locomotion

- Tabular methods are “exact”: you can obtain the **optimal policy**
- ...but they cannot be used for large scale problem
- DQN solved this for discrete actions problems (e.g. videogames)
- PPO did an additional step onwards, and made continuous actions problems tractable

Tomorrow's Outline

- Environments
- Observations
- Reward Functions
- From Simulators to Real World



<https://forms.gle/c8sD2uuYx3T5Yhp69>

Any question, or feedback **on this lesson** is welcome!
Completely anonymous!